

Competitive Programming

Section 13 Editorial: Offline Processing

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	3
2	A. Coordinate Compression	4
2.1	Restatement	4
2.2	Constraints and traps	4
2.3	Observations	4
2.4	Sample walk-through	4
2.5	Algorithm	5
2.6	Correctness proof	5
2.7	Complexity	5
2.8	Full C++ solution	5
2.9	Full Python solution	6
2.10	Common wrong answers	7
2.11	Tests that matter	7
3	B. Range Count At Most	8
3.1	Restatement	8
3.2	Constraints and traps	8
3.3	Observations	8
3.4	Fenwick reminder	9
3.5	Sample walk-through	9
3.6	Algorithm	10
3.7	Correctness proof	10
3.8	Complexity	10
3.9	Full C++ solution	11
3.10	Full Python solution	12
3.11	Common wrong answers	14
3.12	Tests that matter	14
4	C. Distinct Range Queries	15
4.1	Restatement	15
4.2	Constraints and traps	15
4.3	Observations	15
4.4	Sample walk-through	15
4.5	Algorithm	16
4.6	Correctness proof	16
4.7	Complexity	17
4.8	Full C++ solution	17
4.9	Full Python solution	19
4.10	Common wrong answers	20
4.11	Tests that matter	20
5	Stress testing offline solutions	21
6	What to carry forward	22

1 Editorial overview

This section introduces the standard offline workflow:

```
store queries
sort them by a useful key
move a data structure monotonically
write answers by original id
print in original order
```

The local problems use exact outputs, but the underlying patterns are the same as many Codeforces, AtCoder, CSES, and ICPC-style range query tasks.

2 A. Coordinate Compression

2.1 Restatement

Given an array, replace each value by its zero-based rank among all distinct values in that same array. Equal values must get equal ranks, and smaller values must get smaller ranks.

2.2 Constraints and traps

Values can be huge or negative, so they cannot be used directly as array indices.

Common traps:

- assigning a new rank to every occurrence instead of every distinct value;
- sorting the original array and losing original order;
- using one-based ranks when the statement asks for zero-based ranks.

2.3 Observations

Only relative order and equality matter.

Sort the distinct values:

```
values = sorted(unique(a))
```

Then each original value's rank is its index in `values`.

2.4 Sample walk-through

Array:

```
100 50 100 -5 50 7
```

Distinct sorted values:

```
-5 7 50 100
```

Ranks:

```
-5 -> 0
7 -> 1
50 -> 2
100 -> 3
```

Output:

```
3 2 3 0 2 1
```

2.5 Algorithm

```
values = sorted unique copy of a
for each x in a:
    output lower_bound(values, x)
```

In Python, building a dictionary from value to rank is simpler.

2.6 Correctness proof

Lemma 1. Equal original values receive equal ranks.

Proof. Equal values map to the same entry in the sorted distinct list, so the algorithm returns the same index for all equal occurrences. $\square$

Lemma 2. If $x < y$, then $\text{rank}(x) < \text{rank}(y)$.

Proof. The sorted distinct list is in increasing order. Therefore x appears before y , and its index is smaller. $\square$

Theorem. The algorithm outputs exactly the required coordinate compression.

Proof. By Lemma 1, equality is preserved. By Lemma 2, order is preserved. The ranks are zero-based indices in the distinct sorted list, so they are exactly the required ranks. $\square$

2.7 Complexity

```
Time:  $O(n \log n)$ 
Memory:  $O(n)$ 
```

2.8 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

vector<int> solve_case(const vector<long long> &a) {
    vector<long long> values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());

    vector<int> ans;
    ans.reserve(a.size());
    for (long long x : a) {
        ans.push_back((int)(lower_bound(values.begin(), values.end(), x) - values.begin()));
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        vector<int> ans = solve_case(a);
        for (int i = 0; i < n; ++i) {
            if (i) cout << ' ';
            cout << ans[i];
        }
        cout << '\n';
    }

    return 0;
}

```

2.9 Full Python solution

```

import sys

def solve_case(a: list[int]) -> list[int]:
    values = sorted(set(a))
    rank = {value: i for i, value in enumerate(values)}
    return [rank[x] for x in a]

```

```
def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        idx += 1
        a = data[idx:idx + n]
        idx += n
        out.append(" ".join(map(str, solve_case(a))))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

2.10 Common wrong answers

- Printing the sorted compressed array instead of original order.
- Compressing each occurrence separately.
- Forgetting negative values.

2.11 Tests that matter

```
all equal values
already sorted values
large negative and positive values
duplicates mixed with unique values
```

3 B. Range Count At Most

3.1 Restatement

For each query $l\ r\ x$, count how many values in $a[l..r]$ are at most x . Queries must be answered in original order.

3.2 Constraints and traps

The direct solution checks every value in every query, which can be $O(nq)$.

Common traps:

- sorting queries but printing in sorted order;
- adding values with $< x$ instead of $\leq x$;
- using zero-based array positions inside a one-based Fenwick tree.

3.3 Observations

If queries are processed in increasing x , then the set of array positions whose values are $\leq x$ only grows.

Sort array values:

```
(value, position)
```

Sort queries by threshold:

```
(x, l, r, original_id)
```

Maintain a Fenwick tree over positions. When a value becomes active, add 1 at its position. Then a query answer is the number of active positions in $[l, r]$.

3.4 Fenwick reminder

The Fenwick tree used here is one-based.

It supports three operations:

```
add(pos, delta):    a[pos] += delta
sum(pos):           a[1] + ... + a[pos]
range_sum(l, r):    sum(r) - sum(l-1)
```

The key expression is:

```
lowbit(i) = i & -i
```

`bit[i]` stores the sum of the block ending at `i` whose length is `lowbit(i)`. For example:

```
bit[6] stores positions [5,6] because lowbit(6) = 2
bit[8] stores positions [1,8] because lowbit(8) = 8
```

For prefix sums, jump left:

```
while idx > 0:
    total += bit[idx]
    idx -= idx & -idx
```

For point updates, jump right:

```
while idx <= n:
    bit[idx] += delta
    idx += idx & -idx
```

Each jump changes a binary block, so both operations take $O(\log n)$.

3.5 Sample walk-through

Array:

```
index: 1 2 3 4 5
value: 4 1 7 3 4
```

Query 2 4 2 is processed when threshold $x = 2$.

Only value 1 at position 2 has been activated, so:

```
range_sum(2,4) = 1
```

Query 1 5 4 is processed after activating positions with values 1,3,4,4:

```
active positions: 1,2,4,5
range_sum(1,5) = 4
```

3.6 Algorithm

```
values = sorted (a[i], i)
queries = sorted (x, l, r, id)
bit = empty Fenwick tree
ptr = 0

for query in queries:
    while ptr < n and values[ptr].value <= query.x:
        bit.add(values[ptr].position, 1)
        ptr += 1
    ans[query.id] = bit.range_sum(query.l, query.r)

print ans in id order
```

3.7 Correctness proof

Lemma 1. Before answering a query with threshold x , the Fenwick tree has 1 exactly at positions whose array values are $\leq x$.

Proof. Values are processed in increasing order. The pointer advances while the next value is $\leq x$, adding exactly those positions. Since queries are also processed in nondecreasing x , no previously added value ever becomes invalid, and no value greater than x has been added. $\square$

Lemma 2. For a query $l \ r \ x$, `bit.range_sum(l,r)` equals the number of values in $a[l..r]$ that are $\leq x$.

Proof. By Lemma 1, active Fenwick positions are exactly positions with values $\leq x$. Summing active positions over $[l,r]$ counts exactly such values inside the query range. $\square$

Theorem. The algorithm returns the correct answer for every query in the original order.

Proof. Lemma 2 proves every computed query answer is correct. Each query stores its original id, and the algorithm writes the result to `ans[id]`. Printing `ans` from left to right therefore restores the original query order. $\square$

3.8 Complexity

Sorting plus Fenwick operations:

```
Time:  $O((n+q) \log n + q \log q)$ 
Memory:  $O(n+q)$ 
```

3.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    vector<int> bit;

    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int idx, int delta) {
        while (idx < (int)bit.size()) {
            bit[idx] += delta;
            idx += idx & -idx;
        }
    }

    int sum(int idx) const {
        int total = 0;
        while (idx > 0) {
            total += bit[idx];
            idx -= idx & -idx;
        }
        return total;
    }

    int range_sum(int left, int right) const {
        return sum(right) - sum(left - 1);
    }
};

struct Query {
    int l;
    int r;
    long long x;
    int idx;
};

vector<int> solve_case(const vector<long long> &a, vector<Query> queries) {
    vector<pair<long long, int>> values;
    values.reserve(a.size());
    for (int i = 0; i < (int)a.size(); ++i) {
        values.push_back({a[i], i + 1});
    }
    sort(values.begin(), values.end());
    sort(queries.begin(), queries.end(), [](const Query &lhs, const Query &rhs) {
        return lhs.x < rhs.x;
    });

    Fenwick bit((int)a.size());
```

```

vector<int> ans(queries.size());
int ptr = 0;
for (const Query &query : queries) {
    while (ptr < (int)values.size() && values[ptr].first <= query.x) {
        bit.add(values[ptr].second, 1);
        ++ptr;
    }
    ans[query.idx] = bit.range_sum(query.l, query.r);
}
return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, q;
        cin >> n >> q;
        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        vector<Query> queries;
        queries.reserve(q);
        for (int i = 0; i < q; ++i) {
            Query query;
            cin >> query.l >> query.r >> query.x;
            query.idx = i;
            queries.push_back(query);
        }
        vector<int> ans = solve_case(a, queries);
        for (int value : ans) {
            cout << value << '\n';
        }
    }

    return 0;
}

```

3.10 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n: int) -> None:
        self.bit = [0] * (n + 1)

```

```

def add(self, idx: int, delta: int) -> None:
    while idx < len(self.bit):
        self.bit[idx] += delta
        idx += idx & -idx

def sum(self, idx: int) -> int:
    total = 0
    while idx > 0:
        total += self.bit[idx]
        idx -= idx & -idx
    return total

def range_sum(self, left: int, right: int) -> int:
    return self.sum(right) - self.sum(left - 1)

def solve_case(a: list[int], queries: list[tuple[int, int, int]]) -> list[int]:
    values = sorted((value, pos) for pos, value in enumerate(a, start=1))
    ordered = sorted((x, l, r, idx) for idx, (l, r, x) in enumerate(queries))
    bit = Fenwick(len(a))
    ans = [0] * len(queries)
    ptr = 0
    for x, l, r, idx in ordered:
        while ptr < len(values) and values[ptr][0] <= x:
            _, pos = values[ptr]
            bit.add(pos, 1)
            ptr += 1
        ans[idx] = bit.range_sum(l, r)
    return ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        q = data[idx + 1]
        idx += 2
        a = data[idx:idx + n]
        idx += n
        queries: list[tuple[int, int, int]] = []
        for _ in range(q):
            l = data[idx]
            r = data[idx + 1]
            x = data[idx + 2]
            idx += 3

```

```
        queries.append((l, r, x))
        out.extend(map(str, solve_case(a, queries)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

3.11 Common wrong answers

- Activating only values $< x$.
- Sorting queries and forgetting ids.
- Using a Fenwick tree over values instead of positions for range queries.

3.12 Tests that matter

```
threshold below all values
threshold above all values
duplicate values equal to threshold
single-element ranges
```

4 C. Distinct Range Queries

4.1 Restatement

For each query $[l, r]$, count how many distinct values appear in that subarray.

4.2 Constraints and traps

The direct `set(a[l-1:r])` solution is too slow for large total input.

Common traps:

- leaving multiple occurrences of the same value active;
- sorting queries by l instead of r for this method;
- forgetting to remove the old latest occurrence.

4.3 Observations

Process positions from left to right. For each value, maintain only its latest occurrence as active.

When processing position i with value v :

```
if v appeared before at position p:
    remove marker at p
add marker at i
latest[v] = i
```

Now consider any query $[l, r]$ after all positions up to r have been processed. A value appears in $[l, r]$ if and only if its latest occurrence among positions $1..r$ is at least l .

So the answer is:

```
number of active markers in [l,r]
```

Sort queries by right endpoint r .

4.4 Sample walk-through

Array:

```
1 2 1 3 2 4 1
```

After processing $r = 5$:

```
value 1 latest at 3
value 2 latest at 5
value 3 latest at 4
```

Markers:

```
index: 1 2 3 4 5
mark:  0 0 1 1 1
```

Query $[2,5]$ has answer:

```
sum markers in  $[2,5] = 3$ 
```

Values are $\{1,2,3\}$.

4.5 Algorithm

```
sort queries by r
bit = empty Fenwick tree
last = empty map
ptr = 0

for query in sorted queries:
    while ptr < query.r:
        ptr += 1
        value = a[ptr]
        if value has previous position:
            bit.add(previous_position, -1)
        bit.add(ptr, +1)
        last[value] = ptr
    ans[query.id] = bit.range_sum(query.l, query.r)

print ans in original order
```

4.6 Correctness proof

Lemma 1. After processing positions $1..r$, the Fenwick tree contains exactly one active marker for each distinct value in the prefix: the latest occurrence of that value.

Proof. When a value appears for the first time, the algorithm adds a marker at its position. When it appears again, the algorithm removes the previous marker and adds a marker at the new position. Therefore exactly the latest occurrence remains active. $\square$

Lemma 2. For a query $[l, r]$, after processing positions through r , a distinct value appears in $a[l..r]$ if and only if its active marker lies inside $[l, r]$.

Proof. If a value appears in $a[l..r]$, then its latest occurrence within the prefix $1..r$ is at least l , so its marker lies in $[l, r]$. If its marker lies in $[l, r]$, then that value clearly appears in the query range. $\square$

Theorem. The algorithm answers every distinct range query correctly and in original order.

Proof. Queries are processed after the pointer reaches their right endpoint. By Lemma 2, summing active markers in $[l, r]$ gives exactly the number of distinct values in that range. Writing to `ans[id]` restores original query order. $\square$

4.7 Complexity

Each position is added once and possibly removed once. Each query does one range sum.

```
Time:  $O((n+q) \log n)$   
Memory:  $O(n+q)$ 
```

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    vector<int> bit;

    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int idx, int delta) {
        while (idx < (int)bit.size()) {
            bit[idx] += delta;
            idx += idx & -idx;
        }
    }

    int sum(int idx) const {
        int total = 0;
        while (idx > 0) {
            total += bit[idx];
            idx -= idx & -idx;
        }
        return total;
    }

    int range_sum(int left, int right) const {
        return sum(right) - sum(left - 1);
    }
}
```

```

};

struct Query {
    int l;
    int r;
    int idx;
};

vector<int> solve_case(const vector<long long> &a, vector<Query> queries) {
    sort(queries.begin(), queries.end(), [](const Query &lhs, const Query &rhs) {
        return lhs.r < rhs.r;
    });

    Fenwick bit((int)a.size());
    unordered_map<long long, int> last;
    vector<int> ans(queries.size());
    int ptr = 0;
    for (const Query &query : queries) {
        while (ptr < query.r) {
            ++ptr;
            long long value = a[ptr - 1];
            auto it = last.find(value);
            if (it != last.end()) {
                bit.add(it->second, -1);
            }
            bit.add(ptr, 1);
            last[value] = ptr;
        }
        ans[query.idx] = bit.range_sum(query.l, query.r);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, q;
        cin >> n >> q;
        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        vector<Query> queries;
        queries.reserve(q);
        for (int i = 0; i < q; ++i) {
            Query query;
            cin >> query.l >> query.r;
            query.idx = i;
            queries.push_back(query);
        }
    }
}

```

```

    }
    vector<int> ans = solve_case(a, queries);
    for (int value : ans) {
        cout << value << '\n';
    }
}

return 0;
}

```

4.9 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n: int) -> None:
        self.bit = [0] * (n + 1)

    def add(self, idx: int, delta: int) -> None:
        while idx < len(self.bit):
            self.bit[idx] += delta
            idx += idx & -idx

    def sum(self, idx: int) -> int:
        total = 0
        while idx > 0:
            total += self.bit[idx]
            idx -= idx & -idx
        return total

    def range_sum(self, left: int, right: int) -> int:
        return self.sum(right) - self.sum(left - 1)

def solve_case(a: list[int], queries: list[tuple[int, int]]) -> list[int]:
    ordered = sorted((r, l, idx) for idx, (l, r) in enumerate(queries))
    bit = Fenwick(len(a))
    last: dict[int, int] = {}
    ans = [0] * len(queries)
    ptr = 0
    for r, l, idx in ordered:
        while ptr < r:
            ptr += 1
            value = a[ptr - 1]
            if value in last:
                bit.add(last[value], -1)
            bit.add(ptr, 1)
    return ans

```

```

        last[value] = ptr
        ans[idx] = bit.range_sum(l, r)
    return ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        q = data[idx + 1]
        idx += 2
        a = data[idx:idx + n]
        idx += n
        queries: list[tuple[int, int]] = []
        for _ in range(q):
            l = data[idx]
            r = data[idx + 1]
            idx += 2
            queries.append((l, r))
        out.extend(map(str, solve_case(a, queries)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

4.10 Common wrong answers

- Keeping every occurrence active.
- Removing the new occurrence instead of the old occurrence.
- Answering a query before processing up to its right endpoint.
- Printing sorted-query order.

4.11 Tests that matter

```

all values equal
all values distinct
single-position queries
queries with same right endpoint
large or negative values

```

5 Stress testing offline solutions

Offline algorithms are vulnerable to ordering bugs. Brute force small cases:

Range Count At Most:

scan `a[l..r]` for every query and count values `<= x`.

Distinct Range Queries:

build `set(a[l..r])` for every query.

Coordinate Compression:

compare against `sorted(set(a))`.

Randomize query order heavily. A solution that forgets answer restoration often passes sorted queries but fails random query order.

6 What to carry forward

Offline processing is a way of buying monotonicity.

You choose a sort key so that something only moves forward:

- threshold grows;
- right endpoint grows;
- event coordinate grows;
- Mo's algorithm block order changes endpoints cheaply.

Then you keep the final output honest by storing original ids. This idea will reappear in graph algorithms, data structures, computational geometry, and advanced range-query techniques.